Air University, Aerospace and Aviation Campus, Kamra
Department of Computer Science
BS. Cyber Security

CS112L: Object Oriented Programming Lab

Semester Lab Project Report

BSCYS – IIA

Project Title	Library Management System
Lab No.	12
Semester	BS(CyS) – IIA (Spring 2025)
Project ID	OOP-106
Name of Group Members	1. Syed Yaseen Shah (245062) (leader) Sign_____ 2. Furqan Ali (245046) Sign_____ 3. Muhammad (245080) Sign_____ 4. Irfan Iqbal (245052) Sign_____
Date of Submission	Monday, May 26, 2025
Date of Presentation	Monday, May 26, 2025
Submitted To	**Waqar Azeem** Lab Engineer (Cyber Security) Department of Computer Science, AACK
Marks Obtained	_____ **/ 30**
Remarks & Signature by Instructor:	

1 Table of Contents

2 Introduction: 3
2.1 Overview of the Project: 3
2.2 Purpose of the Project: 4
2.3 Objectives of the Project: 4
3 Scope of Work 5
3.1 Overview of Features and Functionality: 5
3.1.1 Core Features 5
3.1.2 Book Management Module 5
3.1.3 Student/Member Management Module 5
3.1.4 Borrow/Return Module 5
3.1.5 Account and Utility Features 6
3.1.6 Technical Highlights 6
4 Experimental Setup: 6
4.1 Development Environment: 6
4.2 Step-by-Step Configuration or Deployment Methods: 6
5 Implementation Details 8
5.1 Code Structure: 8
5.2 Key Code Snippets: 9
5.3 Challenges During Implementation: 10
5.4 Working Methodology: 10
6 Experimentation & Results: 11
6.1 Screenshots 12
7 Conclusion / Lesson Learnt: 15
8 References: 15
9 Appendix : 16
9.1 Sample SQL Code: 16
9.2 Error Messages: 16
9.3 Database Structure Overview: 18

Figure 1: Key Code Snippets of DBConnection....9
Figure 2: Key Code Snippets of login Button validation Logic....10
Figure 3 : Flow Chart....11
Figure 4: Login Page of Library Management System....12
Figure 5: Dashboard of Library Management System....13
Figure 6: Registration of Member Form....14
Figure 7: Member Details page....14
Figure 8: Invalid credentials Error....17
Figure 9: Searching Invalid Book Error....17
Figure 10: Deleting Member (Borrowed Book) Error....18
Figure 11: Database Structure Showing....18

2 Introduction:

2.1 Overview of the Project:

This project focuses on the development of a comprehensive Library Management System using Java as the core programming language. The application features a graphical user interface (GUI) built with Java Swing and integrates with a MySQL database hosted via XAMPP for persistent data storage. The system is designed to streamline the daily operations of an academic library, reducing manual effort and increasing operational efficiency.

The system supports essential features such as user authentication, book management, member registration, book issue/return transactions, viewing borrowed records, report generation, and account management. Each of these functions is accessible through a centralized dashboard after successful login authentication.

This system acts as a robust automation tool for libraries that otherwise rely on paper-based records or disorganized digital files. Through its modern GUI, it allows users to interact with various components like JFrame, JTable, JTextField, and JButton, making it intuitive and responsive.

Importantly, the system includes features such as:

- Form validations to prevent incomplete or invalid data entry
- Search and filtering capabilities for quick record access
- Error handling using Java's try-catch mechanisms
- Modular class-based design for each functional unit (e.g., AddBook, ManageStudents, IssueBook)

By automating book inventory and membership records, the system reduces redundancy, improves accuracy, and ensures secure storage of all transactions.

2.2 Purpose of the Project:

The primary purpose of this project is to help students bridge the gap between theoretical knowledge of object-oriented programming (OOP) and its practical application in real-world software systems. Through this project, students engage with key development stages—design, implementation, testing, and deployment—of a working application that simulates a real-world use case.

Specifically, the project enables:

- Real-time problem-solving while designing GUI and backend connections
- Experience with database operations including SQL CRUD queries and relational schema design
- Practical implementation of core OOP principles like encapsulation, inheritance, and polymorphism
- Exposure to file structures, form validation, data flow control, and multi-class modular coding

It also prepares students for future roles in software engineering by reinforcing:

- Good coding practices (naming conventions, documentation)
- Modular and reusable architecture
- Integration between front-end (GUI) and back-end (database)
- Debugging, testing, and iterative improvement processes

2.3 Objectives of the Project:

- Understand object-oriented programming concepts in Java including classes, inheritance, and polymorphism
- Learn GUI development using Java Swing components
- Perform CRUD operations using MySQL with JDBC connectivity
- Apply modular code design to separate concerns and promote maintainability
- Implement proper exception handling to manage runtime errors effectively
- Gain experience with the complete software development lifecycle from planning to deployment

3 Scope of Work

3.1 Overview of Features and Functionality:

The Library Management System is a desktop-based application built using Java Swing for the GUI and MySQL for backend data storage. It implements a modular structure adhering to Object-Oriented Programming (OOP) principles. Below is a breakdown of its core functionalities:

3.1.1 Core Features

- Authentication System: Ensures only authorized users can access the system. The login form verifies admin credentials before granting access to the dashboard.
- Dashboard Interface: Acts as a central hub from which all other modules are accessed via clearly labeled buttons and panels.

3.1.2 Book Management Module

- Add Book: Input book details like title and author.
- Delete Book: Remove outdated or incorrect book entries.
- Search Book: Search books by ID, title, or author using dynamic filtering.

3.1.3 Student/Member Management Module

- Register Member: Add new student/member information including ID, name, department, and contact.
- View Member Details: Allows retrieval and display of individual student profiles.
- Discard Membership: Permanently remove a student/member from the system.
- Delete Member: Back-end operation for purging member data.

3.1.4 Borrow/Return Module

- Issue Book: Lend books to members with timestamp logging.
- Return Book: Accept returns and update availability in inventory.
- View Borrowed Records: Track which books are currently borrowed by which members.
- View Fine Report (Optional): Calculate and display fine reports based on due dates.

3.1.5 Account and Utility Features

- Change Password: Allows the admin to update login credentials.
- Logout: Securely exits the current session and returns to login screen.

3.1.6 Technical Highlights

- MySQL Integration: All records are stored persistently using JDBC connections to MySQL.
- Swing-Based GUI: Uses JFrame, JPanel, JButton, JLabel, JTable, and more to provide an interactive and responsive user experience.
- Data Validation: Ensures no form can be submitted with missing or incorrect data. Validations are handled through both Java logic and GUI warnings.
- SQL Query Optimization: CRUD operations are efficiently handled using parameterized SQL statements to prevent injection and optimize performance.

4 Experimental Setup:

4.1 Development Environment:

Programming Language: Java
GUI Library: Java Swing
Database: MySQL via XAMPP
IDE: Visual Studio Code
Java Development Kit (JDK): Version 17 or later
JDBC for connecting Java with MySQL

4.2 Step-by-Step Configuration or Deployment Methods:

- Install Java JDK

Download and install the latest version of the Java Development Kit (JDK) from Oracle's official website.

After installation, set up the environment variable JAVA_HOME and add the JDK bin directory to the system's Path variable so Java can run from any terminal or command prompt.

- Install Visual Studio Code (VS Code)

Download and install VS Code from https://code.visualstudio.com.

Install necessary extensions such as "Java Extension Pack" and "Debugger for Java" to support Java development and debugging inside VS Code.

- Set Up XAMPP and Start Services

Install XAMPP from https://www.apachefriends.org.

Launch the XAMPP Control Panel and start Apache and MySQL services.

Click on "Admin" in MySQL to open phpMyAdmin, the browser-based database manager.

- Create MySQL Database Manually

In phpMyAdmin, create a new database named library_management.

Create tables manually by using SQL CREATE TABLE commands or using the phpMyAdmin interface.

Tables include: books, students, borrowed_books, users, etc., with appropriate columns (IDs, names, etc.).

- Open the Java Project in VS Code

Extract the project .zip (if applicable) and open the root folder in VS Code.

Ensure all .java files are visible in the Explorer and that the Java Development Kit is recognized.

- Run the Main Application Class (Login.java)

Identify and open the main class, typically named Login.java or similar.

Right-click inside the file and choose Run Java, or use the terminal to compile and run it.

The login GUI will appear. Enter valid credentials to access the main dashboard.

AIR UNIVERSITY

5 Implementation Details

5.1 Code Structure:

The source code of the Library Management System is organized in a modular and class-based architecture, aligning with the principles of object-oriented programming. Each functionality is encapsulated in a dedicated Java class, making the system easy to navigate, maintain, and extend in the future.

For instance, the Login.java class manages the authentication logic. It presents a user interface for username and password input, and verifies the credentials before allowing access to the main application dashboard. Once authenticated, the user is redirected to Dashboard.java, which acts as the control panel of the system. From there, the user can navigate to various functional modules including book management, student registration, and transaction handling.

The classes AddBook.java and ManageBooks.java are responsible for adding new books and managing existing book records in the library. Similarly, AddStudent.java and ManageStudents.java provide interfaces and backend logic for student registration and modification of student data. The system also includes IssueBook.java and ReturnBook.java, which are dedicated to handling book lending and return operations respectively.

To manage database connectivity, the class DBConnection.java centralizes the logic for establishing a connection between the Java application and the MySQL database. This avoids redundancy and ensures that changes to connection parameters only need to be made in one place. In addition to this, a utils package is included in the project which contains helper classes and utility functions to assist with repeated tasks such as clearing forms or formatting dates.

The application uses Java Swing for its user interface. Components like JTextField, JLabel, JButton, JTable, and JComboBox are used extensively to design user-friendly forms and interactive data views. These components respond to user actions through event listeners that trigger backend logic, such as SQL queries for retrieving or modifying database records.

The application implements robust exception handling. try-catch blocks are placed around all database operations and form validations to prevent the application from crashing due to

unexpected inputs or runtime errors. When an error occurs, meaningful messages are displayed to guide the user.

Form validation is a key part of the system's reliability. Before any data is submitted to the database, it is checked to ensure that required fields are not left empty and that values follow the correct format. For instance, while issuing a book, the system verifies whether the book

exists, is available in stock, and that the student is registered. If any of these checks fail, the operation is aborted with a warning dialog to the user.

This approach not only ensures data consistency and correctness but also protects the system from invalid or incomplete inputs.

5.2 Key Code Snippets:

The following code demonstrates how a database connection is established through the DBConnection class:

```
public class DBConnection {
    public static Connection getConnection() throws SQLException {
        String url = "jdbc:mysql://localhost:3306/library_management";
        String user = "root";
        String password = "";
        return DriverManager.getConnection(url, user, password);
    }
}
```

Figure 1: Key Code Snippets of DBConnection

Another critical snippet illustrates the login validation logic, where user credentials are checked and access is granted based on their validity:

```java
btnLogin.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        String user = txtUsername.getText();
        String pass = new String(txtPassword.getPassword());
        if (user.equals("admin") && pass.equals("admin123")) {
            new Dashboard();
            dispose();
        } else {
            JOptionPane.showMessageDialog(null, "Invalid credentials");
        }
    }
});
```

Figure 2: Key Code Snippets of login Button validation Logic

5.3 Challenges During Implementation:

During development, several technical challenges were encountered and addressed. One of the initial difficulties was ensuring proper setup of the JDBC driver and establishing a reliable connection between the Java application and the MySQL database. Another challenge involved designing an intuitive user interface using Java Swing components, which required careful placement and alignment of elements for usability.

Handling event-driven programming using listeners added complexity, particularly when chaining multiple user actions to database operations. Additionally, debugging SQL syntax and logic errors during the early testing phases was time-consuming. Coordinating navigation between classes, especially while switching views and forms, also required precise control and state management.

5.4 Working Methodology:

The system follows a clean modular methodology where each functional feature is implemented in a separate, self-contained class. This separation of concerns promotes maintainability and scalability. Events initiated by user actions—such as button clicks or form submissions—trigger appropriate functions that process data and interact with the MySQL

database. Real-time data manipulation is achieved using SQL statements executed via JDBC, and updates are reflected immediately in the GUI through Swing components.

This structure ensures that every component in the application interacts seamlessly with both the frontend and backend, offering a smooth and reliable user experience.

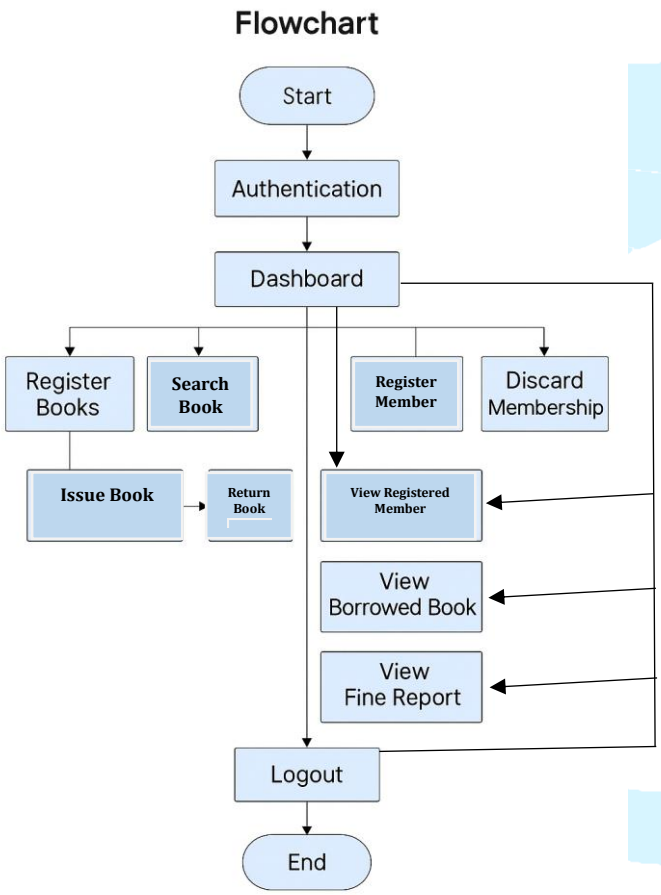


Figure 3 : Flow Chart

6 Experimentation & Results:

The Library Management System was implemented and tested thoroughly, and the outcomes indicate that the application meets its intended functionality across all major modules.

The system correctly handles the addition and deletion of student records. Each student entry includes a unique ID and associated attributes such as name . All student data is validated before submission, ensuring that incomplete or incorrect records cannot be stored in the system.

Data persistence is maintained through MySQL database integration. Once records are added, they remain stored securely and can be accessed, updated, or removed in future sessions. The system ensures that changes in data are reflected immediately in the user interface and underlying database without any errors.

Similarly, book records are managed effectively. The add, delete, and search operations on books were verified through manual testing. The JTable component refreshes dynamically, allowing users to see real-time updates of the library inventory.

Book transactions (issue and return) are also recorded correctly. The system prevents over-issuing of books when quantity is zero, and validates both student and book details before processing a transaction. Returned books are accurately tracked and made available again for issuing.

User interactions are intuitive, and error handling mechanisms are in place to provide feedback for invalid inputs, failed queries, or unauthorized actions. Exception handling ensures the application remains stable under different use conditions.

To support the results of this implementation, selected screenshots from the application are attached. These visuals demonstrate the working of key features while maintaining clarity without overwhelming the documentation.

6.1 Screenshots

Login Page

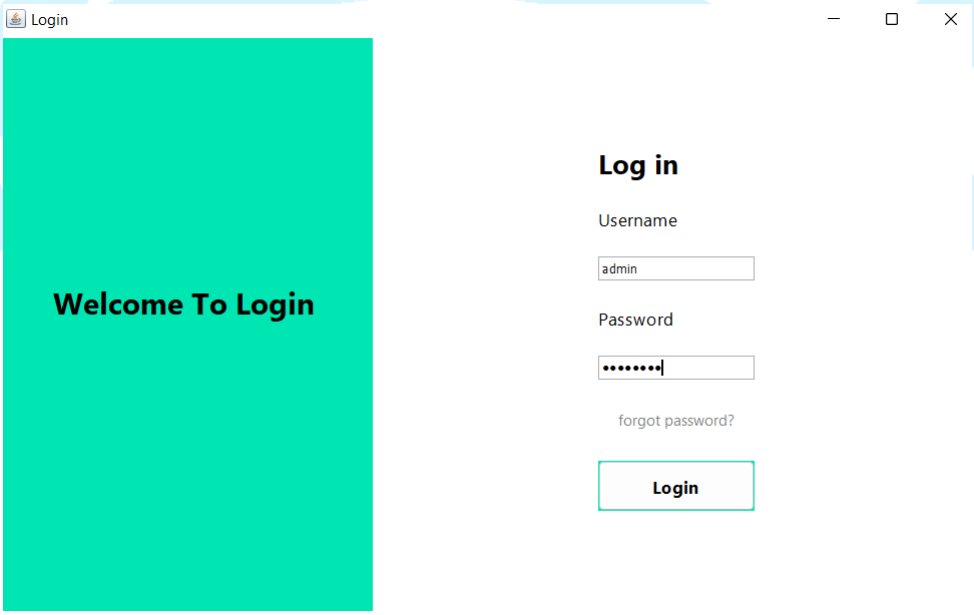


Figure 4: Login Page of Library Management System

Dashboard

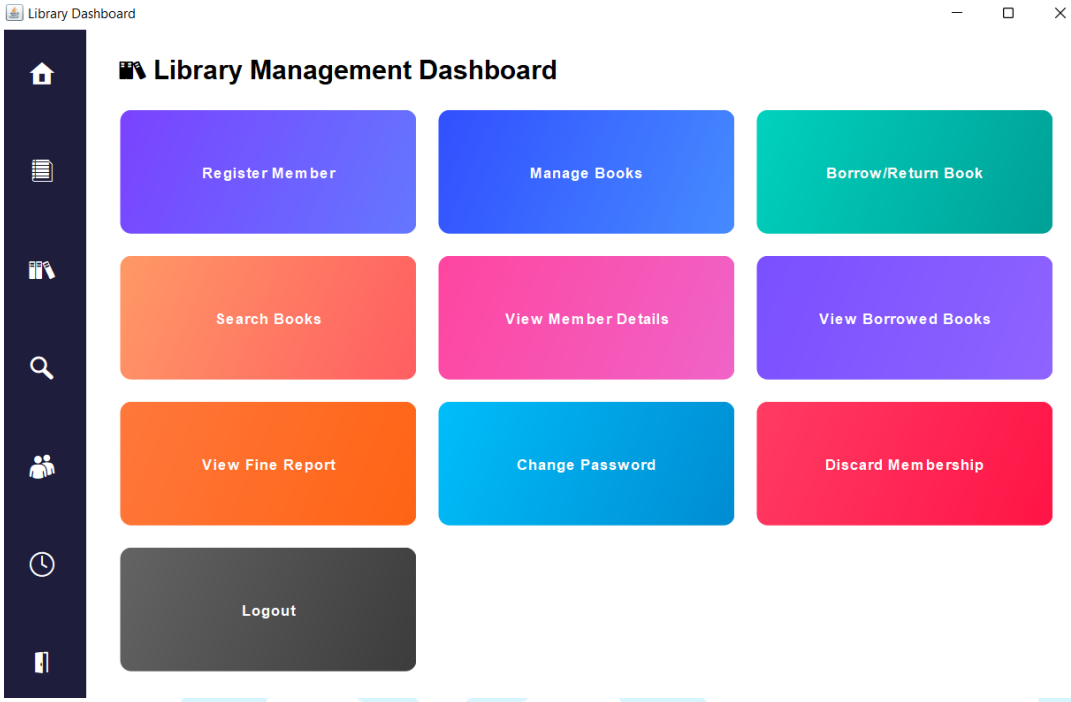


Figure 5: Dashboard of Library Management System

Register Student Form

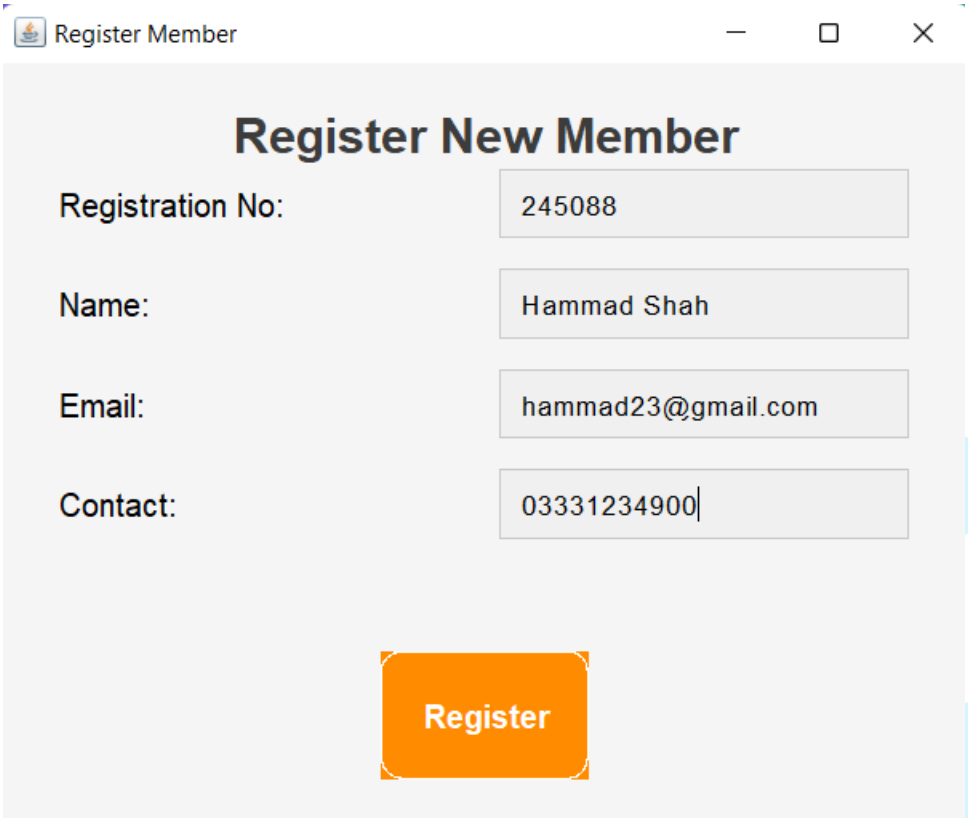
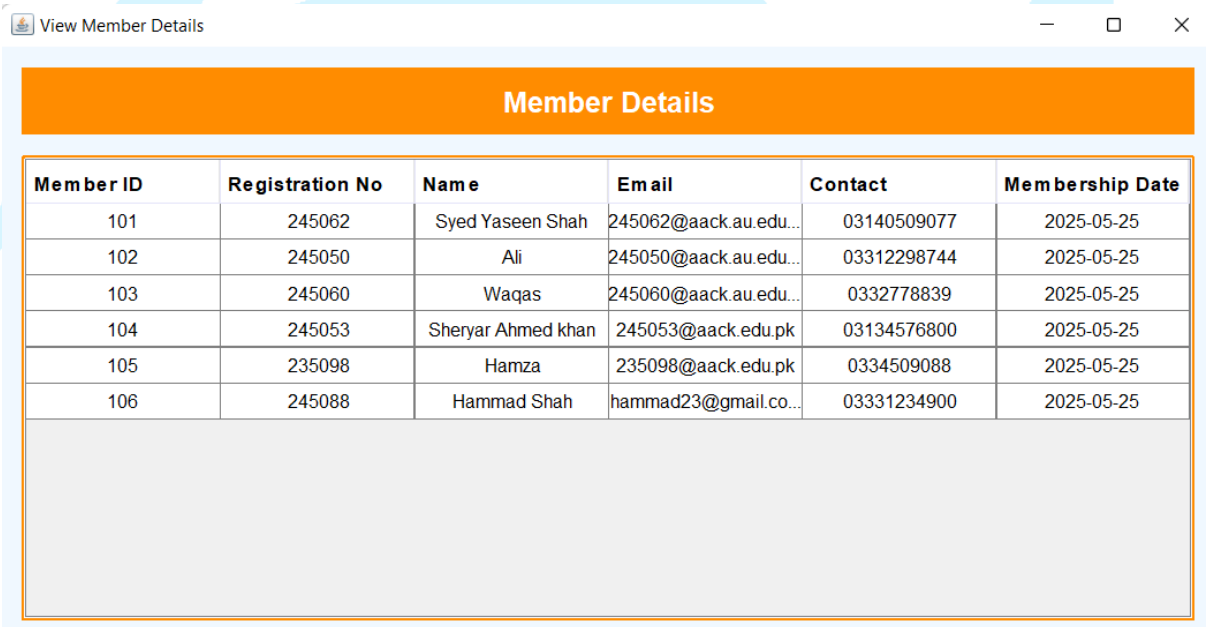


Figure 6: Registration of Member Form

Manage Member Panel



Member ID	Registration No	Name	Email	Contact	Membership Date
101	245062	Syed Yaseen Shah	245062@aack.au.edu...	03140509077	2025-05-25
102	245050	Ali	245050@aack.au.edu...	03312298744	2025-05-25
103	245060	Waqas	245060@aack.au.edu...	0332778839	2025-05-25
104	245053	Sheryar Ahmed khan	245053@aack.edu.pk	03134576800	2025-05-25
105	235098	Hamza	235098@aack.edu.pk	0334509088	2025-05-25
106	245088	Hammad Shah	hammad23@gmail.co...	03331234900	2025-05-25

Figure 7: Member Details page

7 Conclusion / Lesson Learnt:

This project offered comprehensive exposure to the entire software development lifecycle, from planning and design to implementation, testing, and documentation. It served not only as a practical application of object-oriented programming (OOP) principles but also as a real-world simulation of building a complete software system with database integration and a graphical user interface.

One of the most valuable outcomes of the project was the significant improvement in understanding core Java programming. Working with Java Swing for GUI development helped reinforce knowledge of object-oriented design patterns and user interface architecture. By designing forms, handling layouts, and managing event listeners, the project provided a strong foundation in event-driven programming and user interaction.

The integration of the system with a MySQL database deepened the understanding of backend operations. Students gained hands-on experience with establishing and maintaining database connections using JDBC, executing SQL queries, handling result sets, and ensuring data persistence across sessions. This helped bridge the gap between front-end design and back-end logic, encouraging a full-stack perspective.

Another key takeaway was the importance of modular development. The project was structured using a class-based design where each major functionality (e.g., login, book management, student management, issue/return operations) was encapsulated within its own Java class. This approach promoted clean code organization, easier debugging, and future scalability. Debugging and troubleshooting became more manageable through the consistent use of exception handling and form validation across all modules.

Additionally, the project fostered critical thinking and problem-solving skills. Various challenges—such as ensuring correct data validation, handling database errors, and maintaining synchronization between GUI components and backend data—required logical reasoning and creative technical solutions. Working through these issues improved not just technical skills, but also patience, attention to detail, and perseverance.

From a user-experience perspective, the design process emphasized clarity, usability, and functionality. The resulting system is intuitive for the end-user and handles both valid and invalid user behavior gracefully.

Overall, the project helped solidify foundational programming concepts, encouraged adherence to software design principles, and resulted in a fully functional, scalable, and maintainable desktop-based application. It also prepared the students for future projects and real-world software development scenarios by providing them with both confidence and competence.

8 References:

- OpenAI. *ChatGPT (GPT-4) Conversational AI*, OpenAI, 2025.
  Used to assist in Java code generation, JDBC integration guidance, Swing GUI structuring, and technical report writing.
  Available at: https://chat.openai.com
- YouTube. *Video Tutorials on Java Swing, JDBC, and Library Management System*.
  YouTube tutorials helped in understanding how to set up and connect a Java application to a MySQL database using JDBC.
  They explained step-by-step how to write the DBConnection class, how SQL queries interact with Java code, and how to manage data using forms and tables.

- Oracle Corporation. *MySQL 8.0 Reference Manual*.
  Official documentation used to understand SQL syntax, table creation, and relational design best practices.
  Available at: https://dev.mysql.com/doc/

9 Appendix :

9.1 Sample SQL Code:

Include important SQL scripts you created manually, such as:

```
CREATE TABLE students (
  student_id INT PRIMARY KEY,
  name VARCHAR(100),
  registration_date DATE
);
CREATE TABLE books (
  book_id INT PRIMARY KEY,
  title VARCHAR(255),
  author VARCHAR(100),
);
```

9.2 Error Messages:

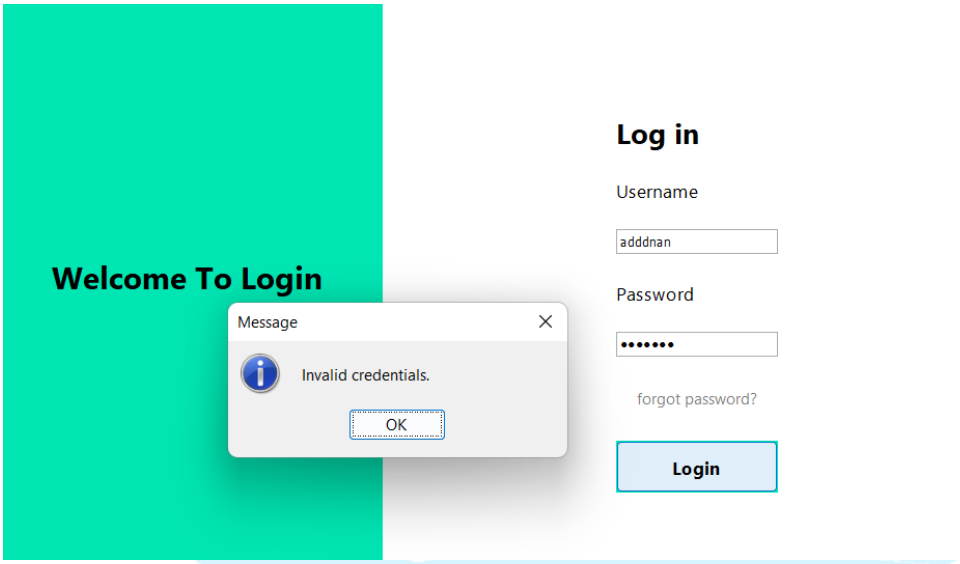


Figure 8: Invalid credentials Error

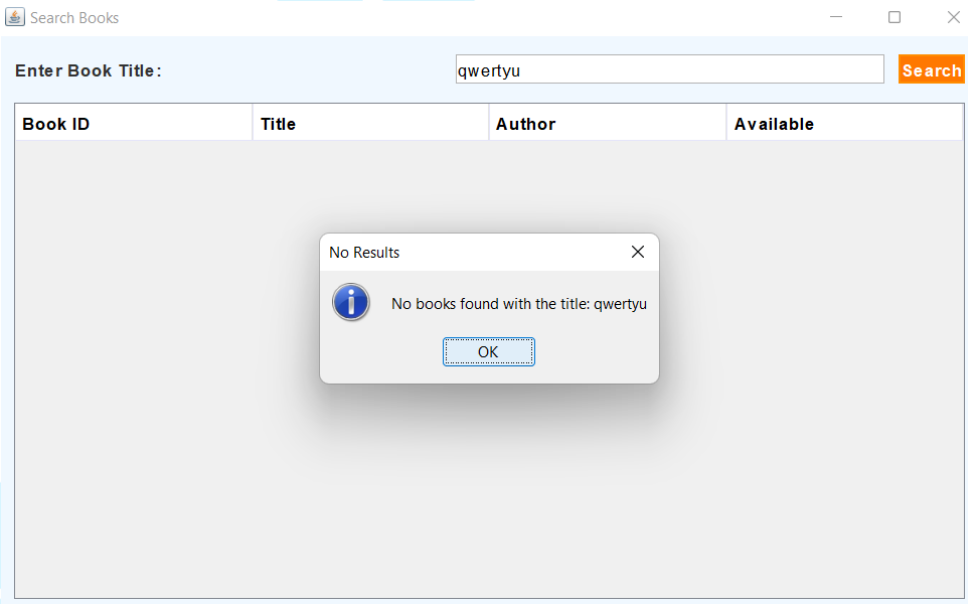


Figure 9: Searching Invalid Book Error

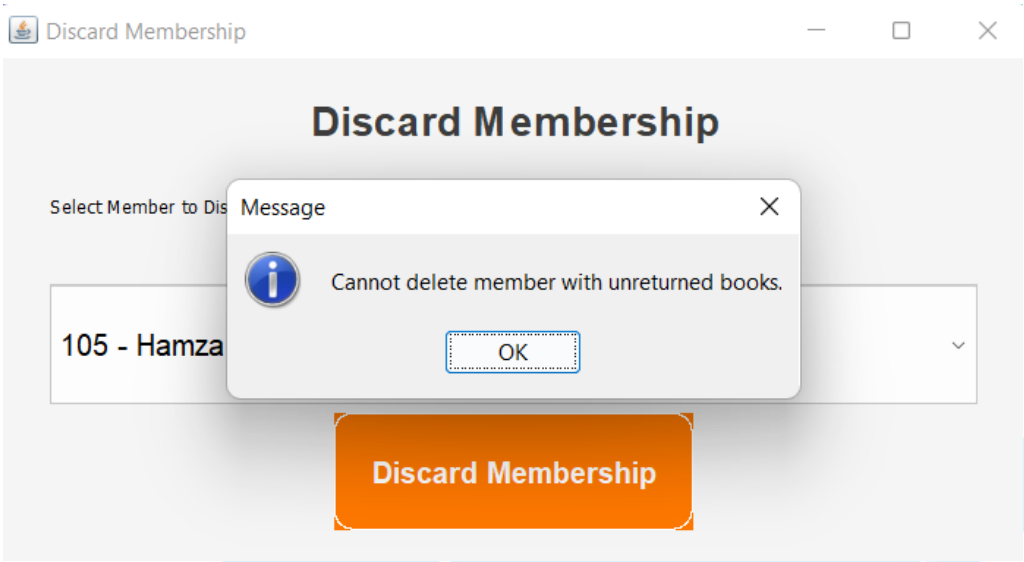


Figure 10: Deleting Member (Borrowed Book) Error

9.3 Database Structure Overview:

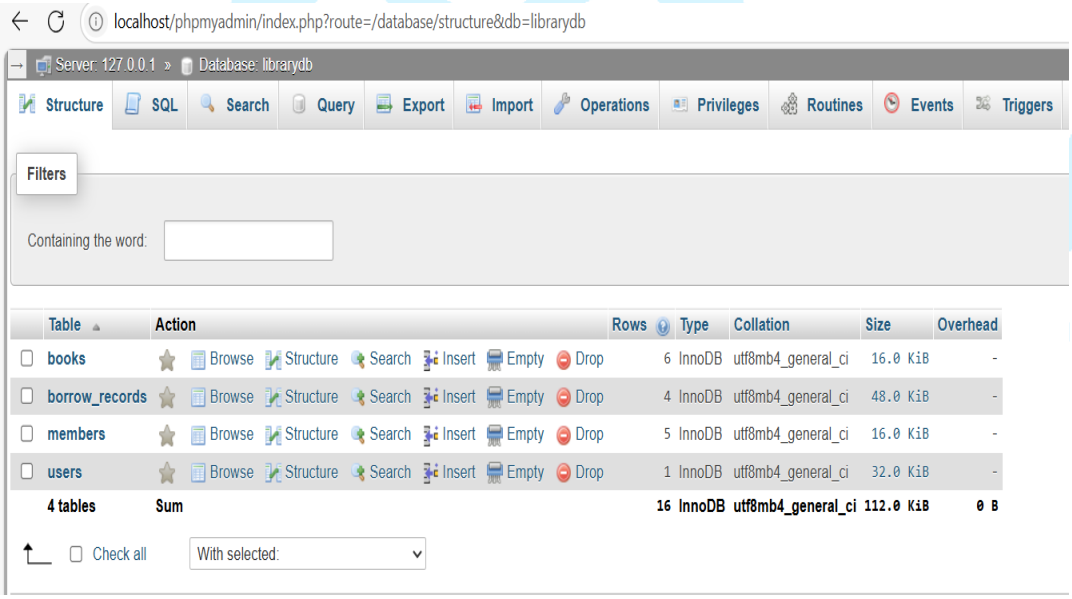


Figure 11: Database Structure Showing